

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: Analyse syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:100

Annexure A

EXPERIMENTS

Code of Conduct:

I HARI VISHNU N / 192524319 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

EXPERIMENTS

Q1. You are developing a compiler-based desk calculator that evaluates arithmetic expressions using syntax-directed definitions (SDD).

The system should:

- (i) Accept an arithmetic expression with operators +, -, *, / and parentheses
- (ii) Use syntax-directed evaluation to compute values based on operator precedence
- (iii) Optimize evaluation by reducing unnecessary temporary computations
- (iv) Display the final result of the expression

Demonstrate how SDD rules (synthesized attributes) are used for expression evaluation.

SDD

Grammar with semantic rules:

$E \rightarrow E + T \quad \{ E.val = E1.val + T.val \}$

$E \rightarrow E - T \quad \{ E.val = E1.val - T.val \}$

$E \rightarrow T \quad \{ E.val = T.val \}$

$T \rightarrow T * F \quad \{ T.val = T1.val * F.val \}$

$T \rightarrow T / F \quad \{ T.val = T1.val / F.val \}$

$T \rightarrow F \quad \{ T.val = F.val \}$

$F \rightarrow (E) \quad \{ F.val = E.val \}$

$F \rightarrow \text{digit} \quad \{ F.val = \text{digit} \}$

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
char expr[100];
int pos = 0;
/* Function declarations */
double E();
double T();
double F();

/* E -> E + T | E - T | T */
double E()
{
    double val = T();
```

```

while (expr[pos] == '+' || expr[pos] == '-')
{
    char op = expr[pos++];
    double tval = T();
    if (op == '+')
        val = val + tval;    // E.val = E1.val + T.val
    else
        val = val - tval;    // E.val = E1.val - T.val
}
return val;
}

```

```

/* T -> T * F | T / F | F */

```

```

double T()
{
    double val = F();
    while (expr[pos] == '*' || expr[pos] == '/')
    {
        char op = expr[pos++];
        double fval = F();
        if (op == '*')
            val = val * fval;    // T.val = T1.val * F.val
        else
        {
            if (fval == 0)
            {
                printf("Error: Division by zero!\n");
                exit(1);
            }
            val = val / fval;    // T.val = T1.val / F.val
        }
    }
    return val;
}

```

```

/* F -> (E) | digit */

```

```

double F()
{
    double val;

    /* F -> (E) */
    if (expr[pos] == '(')
    {
        pos++;                // Skip '('
        val = E();            // F.val = E.val

        if (expr[pos] != ')')

```

```

    {
        printf("Error: Missing '\n");
        exit(1);
    }

    pos++;          // Skip ')'
}
/* F -> digit */
else if (isdigit(expr[pos]))
{
    val = 0;

    /* Allows multi-digit numbers */
    while (isdigit(expr[pos]))
    {
        val = val * 10 + (expr[pos] - '0');
        pos++;
    }

    /* F.val = digit */
}
else
{
    printf("Error: Invalid expression!\n");
    exit(1);
}
return val;
}

int main()
{
    double result;
    printf("Enter arithmetic expression: ");
    fgets(expr, sizeof(expr), stdin);
    result = E();
    while (expr[pos] == ' ' || expr[pos] == '\t')
        pos++;
    if (expr[pos] != '\n' && expr[pos] != '\0')
    {
        printf("Error: Invalid expression!\n");
        return 1;
    }
    printf("\nFinal Result = %.2lf\n", result);
    return 0;
}

```

OUTPUT:

Output

Clear

Enter arithmetic expression: 3+4*5

Final Result = 23.00

=== Code Execution Successful ===

Q2. In a compiler, postfix expressions are evaluated during code generation using stack-based bottom-up evaluation.

Write a C program that:

- (i) Evaluates a postfix expression
- (ii) Demonstrates S-attributed definition evaluation
- (iii) Shows how intermediate results are computed

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <ctype.h>

int stack[100];

int top = -1;

void push(int value)
{
```

```

    stack[++top] = value;
}

int pop()
{
    return stack[top--];
}

int main()
{
    char postfix[100];

    int i, a, b, result;

    printf("Enter postfix expression: ");

    scanf("%s", postfix);

    printf("\nIntermediate Results:\n");

    for (i = 0; postfix[i] != '\0'; i++)
    {
        if (isdigit(postfix[i]))
        {
            result = postfix[i] - '0';

            push(result);

            printf("Push %d\n", result);
        }
        else
        {
            b = pop();

            a = pop();

```

```
switch (postfix[i])
{
    case '+':
        result = a + b;
        break;

    case '-':
        result = a - b;
        break;

    case '*':
        result = a * b;
        break;

    case '/':
        if (b == 0)
        {
            printf("Error: Division by zero\n");
            return 1;
        }
        result = a / b;
        break;

    default:
        printf("Invalid operator\n");
        return 1;
}
```

```

        printf("%d %c %d = %d\n",
               a, postfix[i], b, result);
        push(result);
    }
}
if (top != 0)
{
    printf("Invalid postfix expression\n");
    return 1;
}
printf("\nFinal Result = %d\n", stack[top]);
return 0;
}

```

OUTPUT:

Output	Clear
Enter postfix expression: 34+52-* Intermediate Results: Push 3 Push 4 3 + 4 = 7 Push 5 Push 2 5 - 2 = 3 7 * 3 = 21 Final Result = 21 === Code Execution Successful ===	

Q3.You are designing a parser that uses L-attributed definitions, where inherited attributes are passed from parent to child.

Write a C program that:

- (i).Evaluates a simple expression like $a + b * c$
- (ii) Uses function parameters to simulate inherited attributes
- (iii). Ensures correct precedence handling
- (iv). Demonstrates top-down evaluation logic

Explain how inherited attributes influence computation.

CODE:

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
char expr[100];
```

```
int pos = 0;
```

```
/* Function declarations */
```

```
int E();
```

```
int EPrime(int inherited);
```

```
int T();
```

```
int TPrime(int inherited);
```

```
int F();
```

```
/* E -> T E' */
```

```
int E()
```

```
{
```

```
    int t;
```

```
    t = T();  
    return EPrime(t);  
}
```

```
/* E' -> + T E' | epsilon */
```

```
int EPrime(int inherited)
```

```
{
```

```
    int t;
```

```
    if (expr[pos] == '+')
```

```
    {
```

```
        pos++;
```

```
        t = T();
```

```
        /* Pass inherited value to E' */
```

```
        return EPrime(inherited + t);
```

```
    }
```

```
    return inherited;
```

```
}
```

```
/* T -> F T' */
```

```
int T()
```

```
{
```

```
    int f;
```

```

    f = F();

    return TPrime(f);
}

/* T' -> * F T' | epsilon */
int TPrime(int inherited)
{
    int f;

    if (expr[pos] == '*')
    {
        pos++;

        f = F();

        /* Pass inherited value to T' */
        return TPrime(inherited * f);
    }

    return inherited;
}

/* F -> digit */
int F()
{
    int value;

```

```
    if (isdigit(expr[pos]))
    {
        value = expr[pos] - '0';

        pos++;

        return value;
    }

    printf("Invalid expression\n");

    return 0;
}

int main()
{
    int result;

    printf("Enter expression: ");

    scanf("%s", expr);

    result = E();

    printf("Expression = %s\n", expr);

    printf("Result = %d\n", result);

    return 0;
}
```

OUTPUT:

Output Clear

Enter expression: 2+3*4
Expression = 2+3*4
Result = 14

=== Code Execution Successful ===

Q4. In a compiler, it is important to check whether two type expressions are equivalent.

Write a C program that:

- (i). Accepts two type expressions (e.g., `int`, `float`, `int*`)
- (ii). Compares them for **type equivalence**
- (iii). Displays whether they are:
 - (a). Equivalent
 - (b). Not equivalent

Extend the logic to handle pointer types (`int*`, `float*`).

CODE:

```
#include <stdio.h>

#include <string.h>

int main()
{
    char type1[50], type2[50];

    printf("Enter first type expression: ");
```

```
scanf("%s", type1);  
printf("Enter second type expression: ");  
scanf("%s", type2);  
/* Compare type expressions */  
if (strcmp(type1, type2) == 0)  
{  
    printf("Equivalent\n");  
}  
else  
{  
    printf("Not Equivalent\n");  
}  
return 0;  
}
```

OUTPUT:

Output

Clear

```
Enter first type expression: int  
Enter second type expression: int  
Equivalent
```

=== Code Execution Successful ===

Q5. You are building a semantic analyzer that detects type errors in arithmetic expressions.

Write a C program that:

- (i). Accepts two operands and their types
- (ii). Checks whether the operation (e.g., +, *) is valid
- (iii). Reports:
 - (a). Valid expression
 - (b). Type error (e.g., `int + char*`)

Demonstrate how compilers detect semantic errors during type checking

CODE:

```
#include <stdio.h>
#include <string.h>
int main()
{
    char operand1[20], operand2[20];
    char type1[20], type2[20];
    char op;
    printf("Enter first operand: ");
    scanf("%s", operand1);
    printf("Enter type of first operand: ");
    scanf("%s", type1);
    printf("Enter operator (+, -, *, /): ");
    scanf(" %c", &op);
    printf("Enter second operand: ");
    scanf("%s", operand2);

    printf("Enter type of second operand: ");
    scanf("%s", type2);
```

```

/*
    Arithmetic operations are allowed
    between int and float types.
*/
if ((strcmp(type1, "int") == 0 ||
     strcmp(type1, "float") == 0) &&
    (strcmp(type2, "int") == 0 ||
     strcmp(type2, "float") == 0))
{
    if (op == '+' || op == '-' ||
        op == '*' || op == '/')
    {
        printf("\nValid expression\n");
        if (strcmp(type1, "float") == 0 ||
            strcmp(type2, "float") == 0)
        {
            printf("Result type: float\n");
        }
        else
        {
            printf("Result type: int\n");
        }
    }
    else
    {
        printf("\nInvalid operator\n");
    }
}
else
{
    printf("\nType error\n");
}

```



```
}  
    return 0;  
}
```

OUTPUT:

Output	Clear
<pre>Enter first operand: a Enter type of first operand: int Enter operator (+, -, *, /): + Enter second operand: b Enter type of second operand: int Valid expression Result type: int === Code Execution Successful ===</pre>	

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation